

CSCI 135 Assignment – Classy Fractions

November 8th/10th, 2023

Due Date: Sunday, 11/19/2023 11:59PM (THIS IS A TWO-WEEK LAB)

Points: 100

Topic: Object-Oriented Programming

In this assignment, you will be writing and testing a class to represent fractions and manipulate these fractions using standard operations and operators. In the prior assignment you wrote and tested functions to manipulate fractions represented as tuples containing two integers, so you already have experience with a very similar problem. In that assignment we took a procedural programming approach to this problem. Here we will use an object-oriented approach to encapsulate the associated data and methods into a Python class. While you should directly not use the code from the prior lab as-is to do this assignment, you should be able to use it as a starting point. All functionality should be encapsulated (self-contained) within the class you write.

I am providing you with a starter `ClassyFraction.py` file that implements several of the necessary methods within a corresponding class and defines the member data. This time I do not provide headers describing the remaining functions. It is your responsibility to write the code for and document all of the remaining functions within the class. I am also providing you with a starter `TestClassyFraction.py` file that tests the starter class provided in `ClassyFraction.py`. You should add code to this file to test all the functionality thoroughly, particularly that which you implement. You should write test code not only to test that the functionality performs correctly for expected inputs but so that also it raises exceptions for problematic inputs. This time I will not collect and test your `TestClassyFractions.py` file. It is up to you to thoroughly test your class.

See attached starter files: `ClassyFraction.py`, `TestClassyFraction.py`

Use these as a starting point. Make sure you add/implement all requested items in your final `ClassyFraction.py` file.

Submission:

After following all instructions, please submit your Python script in Moodle. Upload this as an individual file – do not zip it. Be sure the file has an accurate header section including the file name, your name, credits, a description of the file, the class and other useful information such as example usage.

- `ClassyFraction.py`

CSCI 135 Assignment – Classy Fractions

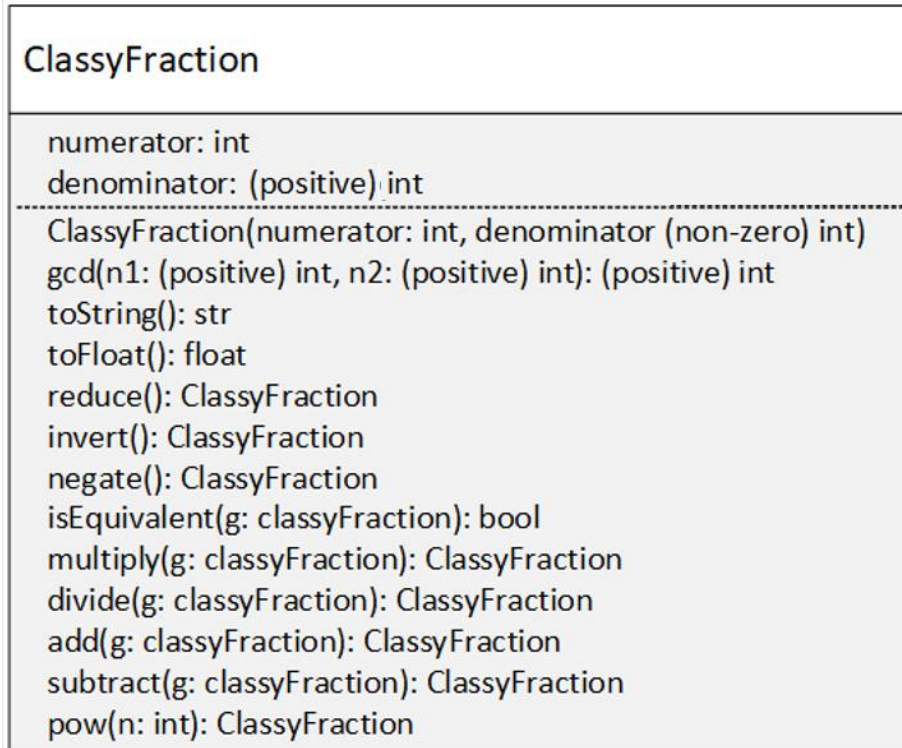
Grading:

This script is worth 100 points. You will be graded according to the following criteria:

	Points
Header Comments for Entire Script / Class	5
Descriptive Comments Throughout Code	5
Header Comments for Class Methods and Comments for Member Data	5
<code>init</code> works in conjunction with constructor and is written correctly	5
<code>gcd()</code> works and written correctly	5
<code>toString()</code> works and written correctly	5
<code>toFloat()</code> works and written correctly	5
<code>reduce()</code> works and written correctly	5
<code>invert()</code> works and written correctly	5
<code>negate()</code> works and written correctly	5
<code>isEquivalent()</code> works and written correctly	5
<code>multiply()</code> works and written correctly	5
<code>divide()</code> works and written correctly	5
<code>add()</code> works and written correctly	5
<code>subtract()</code> works and written correctly	5
<code>pow()</code> works and written correctly	5
Magic methods work and written correctly	20
Total	100

ClassFraction.py

This file includes a single class and associated members that implement the functionality for fractions as described. Pay careful attention to the class starter I provide in the starter file. Here is a UML diagram of the class:



The class shows members in a generic fashion. Additional functionality is necessary to implement magic methods.

The (hidden) member data fields, which I provide in the starter file, are the following:

- ✓ `__numerator`
- ✓ `__denominator`

Tuples are not used in this class. The numerator and denominator are stored individually as `int` objects. The member data is hidden to avoid direct manipulation. As such, it will be set via the construction process and `__init__` function, and cannot be changed directly.

Access to the member data is limited, so we can and should ensure the following:

- The numerator and denominator of a fraction are stored as integers in lowest terms.
- Negative fractions will have a negative numerator and a positive denominator.
- The denominator will always be a positive integer.

Because we assure these things, exception handling becomes easier. It is still necessary.

CSCI 135 Assignment – Classy Fractions

The member methods, including some that I have provided, include the following. To help you, I show the `self` parameter where appropriate. Remember that when you call these methods on objects of the class type you do not include an argument for the `self` parameter. If you call them within other member methods, then you will include the `self` parameter. I also list the magic methods that need to be implemented.

- ✓ `__init__(self, numerator, denominator)`
- ✓ `gcd(n1, n2)`
- ✓ `toString(self)`
- `toFloat(self)`
- ✓ `__reduce__(self)`
- `invert(self)`
- `negate(self)`
- `isEquivalent(self, g)`
- ✓ `multiply(self, g)`
- `divide(self, g)`
- `add(self, g)`
- `subtract(self, g)`
- `pow(self, n)`
- ✓ `__str__(self)`
- ✓ `__repr__(self)`
- `__eq__(self, g)`
- `__add__(self, g)`
- `__sub__(self, g)`
- ✓ `__mul__(self, g)`
- `__truediv__(self, g)`
- `__pow__(self, n)`

`pow()` is a new method here. You did not do this one in the prior lab. The `pow()` method returns a `ClassyFraction` that represents the given `ClassyFraction` raised to the `n`th power where `n` is an integer. `n` may be any integer: positive, negative, or zero. Make sure you handle the various possibilities correctly including cases that should generate an exception.

The magic method `__eq__` should check for equivalence.

Attempts to pass arguments that are not a `ClassyFraction` where one is expected should result in exceptions being thrown.

With the exception of `__init__`, no (public) method should modify the contents of a `ClassyFraction` object. In effect, `ClassyFraction` objects should be immutable. The `__reduce` method is considered private and is only to be called from `__init__`.

CSCI 135 Assignment – Classy Fractions

Helpful hints:

Use the starter files that I provided. Read them, run them, use them as a guide.

Work incrementally. Build and test one method at a time, starting with the easier ones.

Use the methods I provided in the starter as examples of how to write the remaining methods. I carefully chose these so you would have examples that are representative.

Use your work from the prior assignment but realize that changes are needed to turn that work into corresponding class members for this assignment.

Recall how fraction operations work. Addition could be the trickiest of these, but you should have figured that one out in the last lab. Exponentiation may also be tricky. Remember that you are to only implement exponentiation for integer exponents. Floating point methods and types serve no purpose here.

Use the methods you write as helpers in other methods you must write where appropriate. For instance, subtraction can be turned into an addition problem with the right helper functions. Similarly, division can be turned into multiplication. Use your best judgement as to where and if this is appropriate.

Don't just check for proper calculation. Check for invalid inputs and other things that could go wrong. Try to break your code and implement fixes so it won't break when we test it. Generating known exceptions for invalid inputs is part of this. Realize that implementing the data and functionality as a class simplifies and eliminates the need for some of the checks that were necessary in the prior lab.

Upload ClassyFraction.py to Moodle as your submission for this assignment.

EXTENSIONS

You will not submit the answers to these extensions as part of your assignment submission. You should work through and understand them to learn more and practice. You may be held responsible for the content within, so you really should work through them ...

From me:

1. What should the result be for the following calculation?

$$\frac{1}{2} + \frac{1}{2} \times \frac{1}{2}$$

Does your ClassyFraction class calculate this correctly? I.e., does the following yield the correct result?

```
ClassyFraction(1,2) + ClassyFraction(1,2) * ClassyFraction(1,2)
```

Why does this work or not work correctly?